Full Opera Integration Guide

This document describes the current `opera` integration implementation in this repository. It is based on the templates and flow definitions under: `opera/OperaIntegration/agents/OperaAgent/flows/*`

---

1. Overview

The Opera integration provides hotel-focused automation for:

- Availability lookup
- Reservation creation
- Reservation cancellation
- Reservation rescheduling
- Runtime request orchestration and token refresh
- Optional canvas scenario bootstrap
- Optional static data refresh (room types)

Primary agent:

- `OperaAgent`

Primary integration idn:

- `opera_integration`

---

2. Setup and Authorization

The setup entrypoint is `SetupFlow` via event:

- `project_publish_finish` -> `SetupSkill`

2.1 What setup does

`SetupSkill`:

1. Validates target integration idn (`opera_integration`)
2. Initializes project attributes and metadata
3. Ensures HTTP connectors exist:
   - `opera_connector`
   - `opera_token_connector`
4. Ensures a setup persona exists (`setup_persona_id`)
5. Injects booking/availability payload schema attributes
6. Requests OAuth client-credentials token
7. Registers Newo tool definitions for availability/booking/reschedule

2.2 Token flow

`_getTokensSkill` sends:

- `POST {opera_base_url}/oauth/v1/tokens`
- headers include:
  - `x-app-key`
  - `enterpriseId`

    - `Authorization: Basic base64(client_id:client_secret)`
- body:
    - `grant_type=client_credentials`
    - `scope={opera_scope}`

`_getTokensSuccessSkill` stores:

- `opera_access_token`

---

3. Key Attributes

Main project attributes created by setup:

- `opera_base_url`
- `opera_app_key`
- `opera_client_id`
- `opera_client_secret`
- `opera_enterprise_id`
- `opera_hotel_id`
- `opera_scope`
- `opera_access_token`
- `opera_override_agent_attributes`
- `opera_enable_slot_check`
- `opera_enable_booking`
- `opera_enable_cancellation`
- `opera_setup_scenarios`
- `opera_static_data_updated_at`
- `opera_static_data_update_interval`
- `setup_persona_id`

Runtime persona attributes used by flows:

- `opera_availability_status`
- `opera_availability_result`
- `opera_booking_status`
- `bookings` (used by reschedule extraction)

---

4. Flow Capabilities

4.1 AvailabilityFlow

Trigger event:

- `opera_custom_availability_event`

Behavior:

1. Guard by `opera_enable_slot_check`
2. Prevent parallel execution when `opera_availability_status == in_progress`
3. Mark status `in_progress`
4. Extract payload from `arguments.memory` via `_extractPayload`

5. Call Opera availability API via `_checkAvailabilitySkill`
6. Normalize returned offers into a simplified list
7. Publish offers into prompt section:
    - `AvailabilitySlots role="context"`
8. Save simplified offers into persona:
    - `opera_availability_result`
9. Send `urgent_message` instruction for user-facing response

Availability API call:

- `GET /par/v1/hotels/{hotel_id}/availability`
- query includes:
    - `roomStayStartDate`
    - `roomStayEndDate`
    - `roomStayQuantity`
    - `adults`
    - `children`
    - `includeClosedRates=True`
    - `includeLocalRates=True`
    - `limit=10`

Error behavior:

- `ExecuteResultErrorSkill` sets status `error`
- pushes prompt context `There are no available slots.`
- emits `result_error` with `targetAction=check_availability`

4.2 BookingFlow

Trigger event:

- `opera_custom_booking_event`

Behavior:

1. Guard by `opera_enable_booking`
2. Prevent parallel execution when `opera_booking_status == in_progress`
3. Mark status `in_progress`
4. Extract payload from `arguments.memory` via `_extractPayload`
    - Extraction uses `opera_availability_result` as slot source
5. Build Opera reservation payload via `_getBookingPayload`
6. Send reservation request
7. On success, parse reservation id from returned links
8. Push booking context section:
    - `BookingSlots role="context"`
9. Emit `result_success` with:
    - `targetAction=book_slot`
    - `output.success=true`
    - `output.booking_id`

Booking API call:

- `POST /rsv/v1/hotels/{hotel_id}/reservations`

Error behavior:

- `ExecuteResultErrorSkill`:
  - sets `opera_booking_status=error`
  - pushes `Booking was unsuccessful.`
  - emits `result_success` with `success=false` for `book_slot`

4.3 CancellationFlow

Trigger event:

- `convo_agent_cancel_booking`

Behavior:

1. Guard by `opera_enable_cancellation`
2. Reads booking id from payload (`parameters.meta.booking_id`)
3. Sends cancellation request
4. `_cancelAppointmentSuccess` returns `success=true/false` based on reservation status

Cancellation API call:

- `POST /rsv/v1/hotels/{hotel_id}/reservations/{reservation_id}/cancellations`

4.4 RescheduleFlow

Trigger event:

- `opera_custom_reschedule_event`

Behavior:

1. Reads `arguments.memory`
2. Reads existing persona `bookings`
3. Extracts target booking/date changes via `_extractPayload`
4. Builds payload and sends reservation update request
5. On success, emits `result_success` with `targetAction=reschedule_slot`

Reschedule API call:

- `PUT /rsv/v1/hotels/{hotel_id}/reservations/{reservation_id}`

4.5 UtilsFlow (Request Router)

System event entrypoint:

- `opera_execute_request`

Responsibilities:

- Attach Opera headers to outgoing requests
- Route HTTP success/error to integration-level events
- Refresh access token on HTTP 401
- Retry original request (up to attempt limit)

Key events dispatched:

- `opera_result_success`
- `opera_result_error`

4.6 CanvasSetupFlow

Trigger event:

- `gm_workflow_builder_canvas_built`

Purpose:

- Optionally auto-seed booking/cancellation canvas artifacts when `opera_setup_scenarios=True`

Main operations:

- `SetupLibrary` ensures default library items exist
- `SetupCanvas` inserts default canvas items if not already cloned
- emits system events:
  - `gm_canvas_add_library_item_batch`
  - `gm_canvas_delete_item_batch`
  - `gm_canvas_add_item_batch`

4.7 StaticDataFlow

Trigger event:

- `session_started`

Purpose:

- Refresh cached static data by interval

Implemented pieces:

- stale check via `opera_static_data_updated_at` + `opera_static_data_update_interval`
- room type refresh endpoint:
  - `GET /rm/config/v1/hotels/{hotel_id}/roomTypes?limit=1000`
- saves filtered accessible room types into:
  - `opera_room_types`

---

5. Event Map (Operational)

Customer-facing tool events:

- `opera_custom_availability_event`
- `opera_custom_booking_event`
- `opera_custom_reschedule_event`

Internal system bridge events:

- `opera_execute_request`
- `opera_result_success`
- `opera_result_error`

Legacy HTTP callbacks used in Utils/Setup:

- `result_success`
- `result_error`

---

6. Connectors

Defined by setup:

- `opera_connector` (HTTP requests to Opera APIs)
- `opera_token_connector` (token refresh/acquisition path)

---

7. Pre-launch Checklist

1. Confirm credentials and tenant fields:

- `opera_base_url`
- `opera_client_id`
- `opera_client_secret`
- `opera_app_key`
- `opera_enterprise_id`
- `opera_hotel_id`
- `opera_scope`

2. Run setup and verify:

- `opera_access_token` is populated
- `setup_persona_id` exists

3. Confirm capability toggles:

- `opera_enable_slot_check`
- `opera_enable_booking`
- `opera_enable_cancellation`

4. If using canvas bootstrap:

- set `opera_setup_scenarios=True`

5. Validate end-to-end in test suite:

- `pytest -q test/opera`